

1. Описание среды

Цель агента - корректно обрабатывать запросы пользователей системы записи на групповые спортивные занятия.

Среда моделирует простую систему самостоятельной записи клиентов на занятия. Пользователь может записаться на тренировку определённого типа или отменить ранее созданную запись. Агент должен интерпретировать текстовый запрос пользователя, определить корректное действие и при необходимости вызвать соответствующий инструмент. Таким образом, агент решает задачу планирования действий с использованием инструментов, где входом является текстовый запрос пользователя, а выходом - либо текстовый ответ, либо корректный вызов подходящего тула.

В рамках среды моделируются различные ограничения системы записи:

- ограничения по датам и времени занятий
- ограничения по количеству мест (capacity)
- ограничения по типу клиентов
- корректность дат и параметров запроса

Это позволяет проверять способность модели корректно интерпретировать пользовательские запросы, следовать заданным правилам среды, не совершать некорректных действий, избегать галлюцинаций при работе с инструментами.

Среда реализована в виде Python-класса BookingEnv, отнаследованного от ToolEnv, который хранит текущее состояние расписания, текущие параметры и обрабатывает действия агента, делает шаг.

2. Tools

check_availability

check_availability(class_type: "dance" | "yoga" | "stretching", date: datetime)

Инструмент для проверки, можно ли создать запись указанного типа на указанную дату. Проверяет, существует ли тренировка в эту дату + проверяет, не превышено ли допустимое кол-во посетителей (capacity).

Запись может оказаться недоступной по причинам:

```
unknown class type (если модель сгаллюцинировала и передала несуществующий тип занятий)
invalid date
full class capacity
```

И доступной:

```
"It is possible to make an appointment"
```

Ожидается, что агент вызовет этот тул перед тем, как вызвать create_appointment, и после того, как клиент попросит создать запись.

create_appointment

create_appointment(class_type: "dance" | "yoga" | "stretching", date: datetime, sex: "male" | "female", client_id: int)

Инструмент создаёт запись клиента на тренировку. Возвращает: appointment_id: str. Идентификатор формируется как: class_type + date_time + client_id.

cancel_appointment

cancel_appointment(appointment_id: str)

Инструмент проверяет, есть ли такая запись, и отменяет существующую запись. Возвращает True, если запись существовала и была успешно отменена, и False, если запись не существовала. (p.s. в рамках дизайна среды всё так, но в рамках формирования датасета и evaluating/обучения этот тул не использовался (отложен до добрых времён). Его можно было бы применить в задаче с более большой difficulty).

3. Ограничения среды

Чтобы сформировать логику записи в среде были введены фиксированные правила расписания

Тип занятия	Дни месяца	Время	Макс. количество
dance	дни, кратные 5	18:00	15
yoga	дни, кратные 3	19:00	10
stretching	дни, кратные 4	18:30	8

Запись ведётся только на апрель 2026 года (учитывается в формировании дат в датасете + удобно хранить записи). Обрабатываются некорректные даты. Учитываются ограничения вместимости группы. Также еще было ограничение в дизайне, что на stretching могут записываться только female, но в коде этого нет, чтобы не закапываться слишком)

4. Датасет

Для обучения и тестирования агента был создан синтетический датасет сценариев взаимодействия пользователя с системой записи. Каждый пример содержит: *{question_id, user_messages, initial_state, difficulty}*

Созданы train 2000 примеров (траекторий), validation 200 примеров, но val урезается до 50 примеров на Kaggle, train до 250 при GRPO.

user_messages - последовательность сообщений пользователя (многошаговый сценарий),

initial_state - начальное состояние среды,

difficulty - уровень сложности сценария,

5. Уровни сложности

Difficulty 1. Запросы на запись, где дата подходящая. Пример:

```
{
  "question_id": 1,
  "user_messages": [
    "Book dance on 2026-04-35 for client 49. Sex: female.",
    "Yes, confirm the appointment."
  ],
  "initial_state": {
    "schedule": {},
    "appointments": {}
  },
  "difficulty": 1
},
```

Ожидаемый сценарий/поведение агента:

```
User: Book yoga on 2026-04-06 for client 12. Sex: male.
Assistant: check_availability
Assistant: ask confirmation
User: Yes confirm.
Assistant: create_appointment
```

Difficulty 2. Запросы на запись, где дата неподходящая. Затем агент просит поменять, и в след. сообщении от пользователя поступает уже правильная дата. Пример:

```
{
  "question_id": 4,
  "user_messages": [
    "Book stretching on 2026-04-18 for client 65. Sex: female.",
    "Ok, try 2026-04-24 instead.",
    "Yes confirm."
  ],
  "initial_state": {
    "schedule": {},
    "appointments": {}
  },
  "difficulty": 2
},
```

Ожидаемый сценарий/поведение агента:

```
User: Book yoga on 2026-04-05
Assistant: check_availability
Tool: invalid_day
Assistant: explain
User: Ok try 2026-04-06
Assistant: check_availability
Assistant: ask confirmation
User: Yes confirm.
Assistant: create_appointment
```

Difficulty 3. Задизайнено, но не реализовано. Запросы на запись на занятия, где превышено capacity или отмена.

Датасет формировался 70% сложность 1, 30% сложность 2.

6. Оценка действий агента (Reward)

Каждый шаг взаимодействия агента со средой оценивается с точки зрения: корректности вызова инструментов, соблюдения правил среды, отсутствия галлюцинаций (использование только существующих тулов и существующих (упомянутых) entities), достижения цели сценария (создать запись).

Положительная награда:

- достижение цели сценария (создание записи, +1),
- запрос на подтверждение записи у клиента (+ 0.1)

Отрицательная награда:

- некорректный вызов инструмента (невозможно распарсить, -0.1),
- попытка вызвать несуществующий инструмент (-0.2),
- неверные параметры инструмента (-0.05 за каждый лишний аргумент переданный),
- вызов create или cancel (правило среды) до того, как был запрос на подтверждение (-0.2),
- галлюцинации (вымышленные entities, максимум -0.2, рассчитывается пропорционально кол-ву аргументов в tool)

Галлюцинацией считались случаи, когда модель в качестве значения параметра передает сущность, которая ранее не фигурировала в запросе пользователя (неверный тип записи, неверная дата, client_id и т.д.)

7. Prompting

Системный промпт

Описывает тулы, желаемые сценарии поведения

```
self.system_prompt = """You are a helpful assistant for making appointments to sport classes:
'yoga', 'stretching', 'dance'. Client may ask you to create or delete an appointment for sport
class. You should use tools.

Available tools with signature:

1) check_availability(class_type: str, date: str)
Choose it when client asks you to book a class for a certain date. At first, you should check
availability before create a book! If check_availability == True, you should ask a client to
confirm an appointment.

2) create_appointment(class_type: str, date: str, client_id: int, sex: str)
Choose it after client confirms an appointment.

3) cancel_appointment(appointment_id: str)
Choose it when client asks you to cancel an appointment.

Scenarios to communicate with client:
- Client asks to book a sport class with a type and date -> Call tool to Check availability of
this class.
- If class is available -> ask client to confirm an appointment using free text form.
```

```

- If class is not available -> stop invoking tools, say about it to client and provide a reason.
- Client confirms an appointment -> Call tool for Create an appointment with client information.
- Clients asks to cancel an appointment -> Call tool for Cancel.
In other cases, write client only 'To book a class, specify type, date and your sex. To cancel a class, specify ID of appointment.'

Return TOOL_CALL with respective args if it is neccessary or free text.

Example for TOOL_CALL:
user request: Book stretching on 2023-07-25 for client 5. Sex: female
your answer: TOOL_CALL {"name": "check_availability", "args": {"class_type": "stretching", "date": "2023-07-25"}}

Example for free text:
Sorry, but class type yoga for date 2022-09-27 is unavailable.

```

Template для Qwen со вставкой observation

```

messages = [
    {
        "role": "system",
        "content": self.system_prompt,
    },
    {
        "role": "user",
        "content": f"{observation}\n\nRespond with either text or:\n\nTOOL_CALL {{...}} (with respective args)",
    },
]

```

8. Сохранение истории

Агенту в качестве observation передавалось current_observation + все предыдущие состояния, результаты вызовов тулов, предыдущие сообщения агента. Вообще, поддерживалась история.

9. Результаты Prompting

Модель	Success_Rate (запись была создана) Val	Avg_Reward Val
Qwen/Qwen2.5-0.5B-Instruct	0	-0.1
Qwen/Qwen2.5-3B-Instruct	0	-0.476
Qwen/Qwen3-1.7B	0	0.624
Qwen/Qwen3-4B	0	0.638

С помощью промптинга ни на одной модели не удалось получить Success_rate > 0, при len(val_dataset) = 50 значит, что ни один сценарий из 50 не завершился созданием записи. Однако, видно, что при применении

Qwen3 вместо Qwen2.5 Avg_Reward достаточно высоко. Положительная награда присваивается когда достигается цель сценария либо когда агент запросил подтверждение, можно сделать вывод, что хотя бы подтверждение запрашивает)

Примеры сценариев, ответов агента:

p.s. ответы от Qwen3 были рассмотрены только на финальный момент работы - написания отчета

Qwen/Qwen2.5-0.5B-Instruct

Наблюдается, что агент пошел в `check_availability`, получил положительный результат, после этого пошел в `create_appointment`, но получил “*Before creation or cancelling an appointment, it is needed to request client for confirmation*”. После чего хотелось бы видеть текстовое сообщение с запросом о подтверждении, но произошло заикливание, и так пока `steps < max_steps (= 10.)`

```
[ '\nObservation: User request:\nBook dance on 2026-04-40 for client 42. Sex: female.\n\nAgent:
TOOL_CALL {
  "name": "check_availability",
  "args": {
    "class_type": "dance",
    "date": "2026-04-40"
  }
} \n\nObservation: {\ 'valid\': True, \ 'capacity_left\': 15, \ 'time\': \ '18: 00\ ',
\ 'message\': \ 'It is possible to make an appointment for class type: dance, date:
2026-04-40!\ ', \ 'extra_args\': 0
} \n\nAgent: TOOL_CALL {
  "name": "create_appointment",
  "args": {
    "class_type": "dance",
    "date": "2026-04-40",
    "client_id": 42,
    "sex": "female"
  }
} \n\nObservation: Before creation or cancelling an appointment, it is needed to request
client for confirmation.\n\nAgent: TOOL_CALL {
  "name": "check_availability",
  "args": {
    "class_type": "dance",
    "date": "2026-04-40"
  }
} \n\nObservation: {\ 'valid\': True, \ 'capacity_left\': 15, \ 'time\': \ '18: 00\ ',
\ 'message\': \ 'It is possible to make an appointment for class type: dance, date:
2026-04-40!\ ', \ 'extra_args\': 0
} \n\nAgent: TOOL_CALL {
  "name": "create_appointment",
  "args": {
    "class_type": "dance",
    "date": "2026-04-40",
    "client_id": 42,
    "sex": "female"
  }
} \n\nObservation: Before creation or cancelling an appointment, it is needed to request
client for confirmation.\n\nAgent: TOOL_CALL {
  "name": "check_availability",
  "args": {
    "class_type": "dance",
    "date": "2026-04-40"
  }
} \n\nObservation: {\ 'valid\': True, \ 'capacity_left\': 15, \ 'time\': \ '18: 00\ ',
\ 'message\': \ 'It is possible to make an appointment for class type: dance, date:
2026-04-40!\ ', \ 'extra_args\': 0
} \n\nAgent: TOOL_CALL {
  "name": "check_availability",
  "args": {
    "class_type": "dance",
    "date": "2026-04-40"
  }
}
```



```

}\n\nObservation: Invalid tool format, cannot be parsed as JSON\n\nAgent: TOOL_CALL {
  {
    "name": "check_availability",
    "args": {
      "class_type": "dance",
      "date": "2026-04-40"
    }
  }
}
}\n\nObservation: Invalid tool format, cannot be parsed as JSON\n\nAgent: TOOL_CALL {
  {
    "name": "check_availability",
    "args": {
      "class_type": "dance",
      "date": "2026-04-40"
    }
  }
}
}\n\nObservation: Invalid tool format, cannot be parsed as JSON\n\nAgent: TOOL_CALL {
  {
    "name": "check_availability",
    "args": {
      "class_type": "dance",
      "date": "2026-04-40"
    }
  }
}
}\n\nObservation: Invalid tool format, cannot be parsed as JSON\n\nAgent: TOOL_CALL {
  {
    "name": "check_availability",
    "args": {
      "class_type": "dance",
      "date": "2026-04-40"
    }
  }
}
}\n\nObservation: Invalid tool format, cannot be parsed as JSON\n\nAgent: TOOL_CALL {
  {
    "name": "check_availability",
    "args": {
      "class_type": "dance",
      "date": "2026-04-40"
    }
  }
}
}\n\nObservation: Invalid tool format, cannot be parsed as JSON\n'
]

```

Qwen/Qwen3-1.7B

Тут модель повторяла одно и тоже даже после подтверждения записи..

И начала с рассуждения. Такой идентичный ответ повторялся пока макс. кол-во шагов не было достигнуто.

```

['\n\nObservation: User request:\nBook dance on 2026-04-40 for client 42. Sex: female.\n\nAgent: <think>\n0kay, let's see. The user wants to book a dance class on 2026-04-40 for client 42, and they mentioned their sex is female.\n\nFirst, I need to check if the date is valid. Wait, the date here is 2026-04-40. Oh, April 40th doesn't make sense. Dates can't have numbers over 12 months, so 40th day of April is invalid. That's a problem. \n\nSo the check_availability tool would return that the date is invalid. Then,\n\nObservation: User: Yes, confirm the appointment.\n\nAgent: <think>\n0kay, let's break this down. The user wants to book a dance class on 2026-04-40 for client 42, and they're female. First, I need to check if the date is valid. The date given is 2026-04-40. April has 30 days, so 40th day is impossible. That's an invalid date.\n\nSo, the first step is to use the check_availability tool with class_type "dance" and date "2026-04-40". The tool will return\n\nObservation: Message received.\n\nAgent: <think>\n0kay, the user wants to book a dance class on 2026-04-40 for client 42, and they're female. Let me check the date first. April 40th is invalid, right? Because April only has 30 days. So the check_availability tool would return that the date is not available. Then, I need to inform the user that the date is invalid and ask them to confirm if they want to proceed with a different date. But the user hasn't provided a valid date yet. So the correct response is to point out the invalid date and ask\n\nObservation: Message received.\n\nAgent: <think>\n0kay, the user tried to book a dance class on 2026-04-40, but that date is invalid. April only has 30 days, so 40th is impossible. I need to check the availability first. The check_availability tool would return that the date is invalid. Then, I should inform the user about the error and ask them to provide a valid date. Since the user hasn't provided a valid date yet, I can't proceed to create the appointment. So the correct response is to point out the invalid date and ask them to confirm with a valid date.\n\nObservation: Message received.\n\nAgent: <think>\n0kay, let's see. The user wants to book a dance class on 2026-04-40 for client 42, and they're female. First, I need to check if the date is valid. The date is April 40th, which isn't a real date. So the check_availability tool would return that the date is invalid. Then, I should inform the user about the error and ask them to provide a valid date. Since the user hasn't given a valid date yet, I can't proceed to create the appointment. So the correct response is to point out\n\nObservation: Message received.\n\nAgent: <think>\n0kay, the user wants to book a dance class on 2026-04-40. First, I need to check if that date is valid. April has 30 days, so 40th is invalid. The check_availability tool will return that the date is not available. Then, I should inform the user about the error and ask them to provide a valid date. Since the user hasn't given a valid date yet, I can't proceed to create the appointment. So the correct response is to point out the invalid date and ask them to confirm with a valid date.\n\nThe answer\n\nObservation: Message received.\n\nAgent: <think>\n0kay, let's tackle this step by step. The user wants to book a dance class on 2026-04-40 for client 42, and they're female. First, I need to check if the date is valid. The date given is April 40th, which isn't a real date because April only has 30 days. So, the check_availability tool will return that the date is invalid. \n\nSince the date is invalid, I can't proceed to create the appointment. The user hasn't provided a valid date yet, so I need to inform them about\n\nObservation: Message received.\n\nAgent: <think>\n0kay, the user wants to book a dance class on 2026-04-40. First, I need to check if that date is valid. April 40th doesn't exist, so the check_availability tool will return that the date is invalid. The user hasn't provided a valid date yet, so I can't proceed to create the appointment. I should inform them about the invalid date and ask them to provide a valid date. The correct response is to point out the error and ask for a valid date.\n\nSo, the answer is to inform the user that the date is invalid and\n\nObservation: Message received.\n\nAgent: <think>\n0kay, let's see. The user wants to book a dance class on 2026-04-40 for client 42, and they're female. First, I need to check if the date is valid. The date is April 40th, which isn't a real date. April only has 30 days, so 40th is invalid. Therefore, the check_availability tool will return that the date is not available. Since the date is invalid, I can't create the appointment. The user hasn't provided a valid date yet, so I need to inform them\n\nObservation: Message received.\n']

```


Qwen/Qwen3-4B

Такая же ситуация, и ни одного тула вызвано не было, потому что рассуждение было длинным (и не нужным). Собственно, ответы моделей Qwen3 концептуально хуже и бесполезнее, чем Qwen2.5.

[illegible]

10. Попытка обучения с GRPO

Было реализовано: симуляция эпизодов взаимодействия, custom reward функция, интеграция среды с GRPOTrainer. Reward рассчитывался на основе полной траектории действий агента в среде.

Практическая проблема обучения - при запуске обучения на Kaggle на $\text{len}(\text{train}) = 50$ на модели

Qwen/Qwen2.5-0.5B-Instruct было получено следующее оценочное время около 40 часов обучения. В то время как доступное время GPU на платформе составляет около 30 часов. Есть, конечно, вероятность, что я при написании кода допустила ошибку.

Тем не менее, код обучения и reward-функции под GRPO был реализован и он запускается! :)

([grpo_training.py](#)). Из-за ограничений по времени и вычислительным ресурсам добить полноценный запуск RL-обучения не удалось :(

Step	Training Loss	reward	reward_std	completions / mean_length	completions / min_length	completions / max_length	completions / clipped_ratio	completions / mean_terminated_length	completions / min_terminated_length	completions / max_terminated_length
6375	26.10	< 40:15:10	0.00	it/s	Epoch 0.04/3					

И3 grpo_training.py:

```
class RLAgent:
    """Для батчевания"""

    def __init__(self, model, tokenizer):
        self.model = model
        self.tokenizer = tokenizer
```

```

self.device = next(model.parameters()).device

def act_batch(self, observations):
    inputs = self.tokenizer(
        observations,
        return_tensors="pt",
        padding=True,
        truncation=True,
    ).to(self.device)

    with torch.no_grad():
        outputs = self.model.generate(
            **inputs,
            max_new_tokens=CONFIG["max_new_tokens"],
            do_sample=True,
            temperature=CONFIG["temperature"],
            top_k=CONFIG["top_k"],
            pad_token_id=self.tokenizer.pad_token_id,
        )

    responses = []
    for i, prompt in enumerate(observations):
        input_len = inputs["attention_mask"][i].sum().item()
        generated_tokens = outputs[i][input_len:]
        text = self.tokenizer.decode(
            generated_tokens, skip_special_tokens=True
        ).strip()
        responses.append(text)
    return responses

def prepare_grpo_dataset(data):
    """Подготовка фиктивной колонки prompt для соответствия формату GRPOtrainer"""
    rows = []
    for row in data:
        first_msg = row["user_messages"][0]
        prompt = f"""You are a helpful assistant for booking sport classes.

User: {first_msg}

Respond with either text or:

TOOL_CALL {{"name": "...", "args": {{{...}}}}

"""
        rows.append({"prompt": prompt, "scenario": row})
    return Dataset.from_list(rows)

def make_reward_function(model, tokenizer, verifier, dataset):
    """Кастомная ревард-функция, которая оценивает всю траекторию"""
    agent = RLAgent(model, tokenizer)

    def env_reward(prompts, completions, **kwargs):
        rewards = []
        for completion, row in zip(completions, dataset):
            scenario = row["scenario"]
            env = BookingEnv()
            obs = env.reset(scenario)
            actions = [completion]

            obs, reward, done, info = env.step(completion)
            history = [("OBS", obs), ("ACTION", completion)]
            step = 0

            while not done and step < 8:
                prompt = format_history(history)
                action = agent.act_batch([prompt])[0]
                actions.append(action)
                obs, r, done, info = env.step(action)
                reward += r
                history.append(("ACTION", action))
                history.append(("OBS", obs))
                step += 1

            result = verifier.verify_trajectory(BookingEnv(), scenario, actions)
            rewards.append(result["total_reward"])
        return rewards

    return env_reward

def grpo_train_loop(train_data):
    train_dataset = prepare_grpo_dataset(train_data)

```

```
reward_fn = make_reward_function(model, tokenizer, verifier, train_dataset)

trainer = GRPOTrainer(
    model=model,
    processing_class=tokenizer,
    reward_funcs=[reward_fn],
    args=grpo_config,
    train_dataset=train_dataset,
)

trainer.train()
```

11. Размышления, почему промптинг не сработал

Модели маленькие;

Инструкция недостаточно ясна, модель путается, не понимает, заикливается;

Prompting не даёт модели опыта взаимодействия со средой, модель не получает обратной связи о том, какие действия приводят к успеху, а какие к ошибке.